

Unit 3

Q-learning is **tabular** & therefore not scalable
works fine with small state spaces.

State & Action $\rightarrow$ Q table $\rightarrow$ Q value

State $\rightarrow$ Deep Q nn $\rightarrow$ Q value action 1
Q value action 2
Q value action 3.

Deep Q learning uses a neural network to predict Q values. To see a visual game, a CNN is used.

$\rightarrow$ **Preprocessing** - frames are made smaller, grayscale & stacked together to provide **temporal information**.

uses gradient descent to update the weights of the Deep-Q network.

- **Deep Q Algorithm**

1. **Sampling** - perform actions & store the observed experience tuples in a replay mem.

2. **Training** - select a small batch of tuples randomly & learn from this batch using a gradient descent step update.

Deep Q learning could be **unstable** b/c of the combination of bootstrapping & the non-linear nn.

3 solutions for stabilization - **experience replay**, **fixed Q target**, **double deep Q learning**.

Experience Replay

Usually the agent interacts w/ the env, gets experiences, learns from them, then discards the experiences.

- **Catastrophic forgetting** - the nn tends to forget previous experiences as it is learning new ones.
- the **replay buffer** stores experience tuples while interacting & then samples a random batch for training.
 - ↳ also removes correlation in obs. segs. → avoids action vals from converging or diverging catastrophically

Fixed Q-Target

When calculating the TD error (loss) calculate the difference b/w the TD target (Q-target) & the current Q-value (estimation of Q)

Q-Target

$$R_{t+1} + \gamma \max_a Q(S_{t+1}, a)$$

immediate reward discounted estimate of optimal Q-val at next step

Q-Loss

$$(Q\text{-Target}) - Q(S_t, A_t)$$

former Q-val estimate

using the same model w/ the same weights to calculate what Q is & what Q should be!

- ↳ both Q estimation & Q target are moving

use a separate network w/ fixed params. for estimating TD-Target ($\hat{Q}$) → copy the params back to Q every C steps.

Double Deep Q-Learning NNs. (Double DQN)
handles the problem of the overestimation of
Q-values.

accuracy of Q-values depends on the action, $\neq$
what neighbouring states explored

↳ not enough information to take the best
action at the beginning of the training → false positive
if too many false positives → learning bias.

- Use 2 networks to decouple the action selection
from target Q-value generation.

- Use DQN network to select the best action
to take for the next state (action w/ highest Q)

- Use Target network to calculate the target Q
value of taking that action at the next state.

↳ Double DQN helps reduce the overestimation
of Q-values.